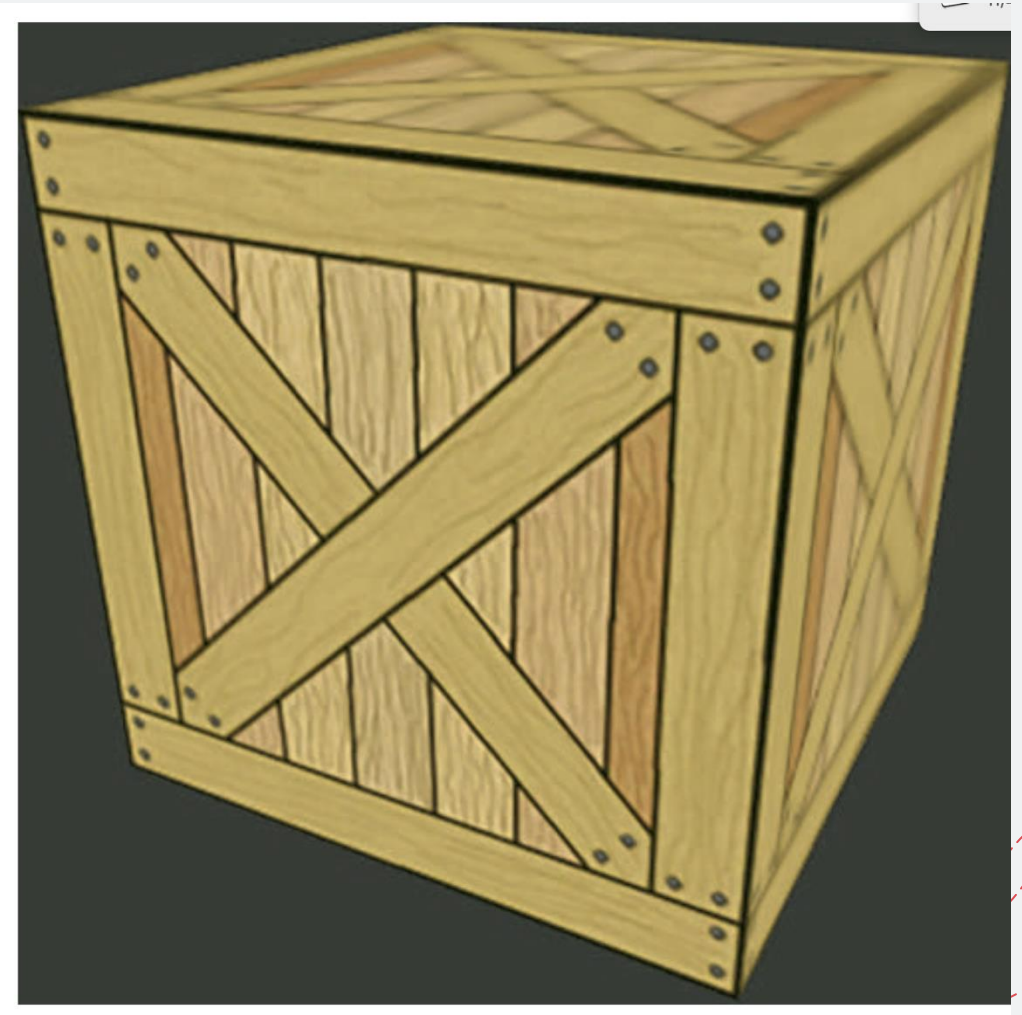
Graphics Programming

Textures in OpenGL

Gongbo “Tony” Liang

Intro to Textures

- Textures – Images applied to surfaces
  - UV coordinates – Assign points from texture to points on surface
- Goals
  - Create a texture class to upload pixel data to the GPU
  - Add UV coordinates to geometric objects
  - Associate textures to uniform variables and use in shaders
  - Render scene including textures

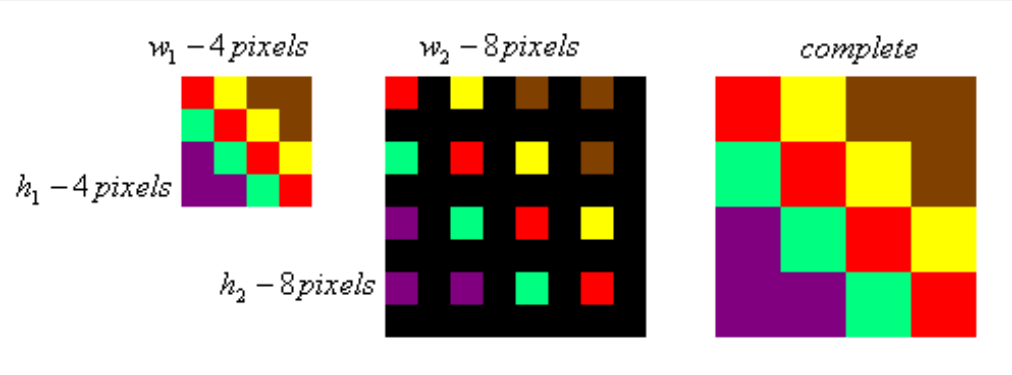


OpenGL Texture Objects

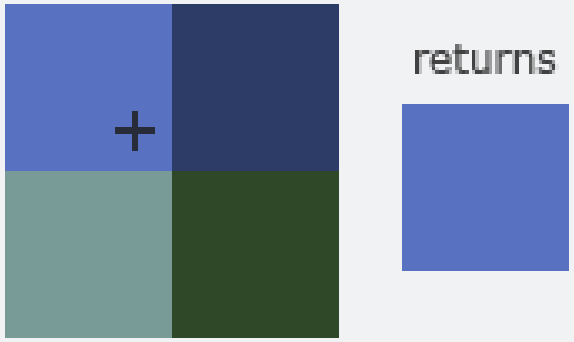
- Data structure that stores pixel data from an image
- Also stores information about which algorithms are used to determine the colors of surface the texture is applied to
  - Require computation when texture and surface are different sizes
    - Magnification and Minification filters

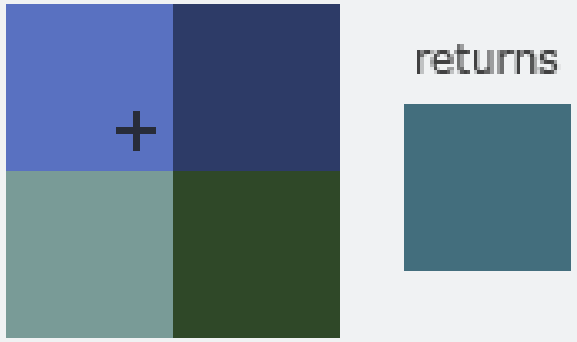
Scaling Image Data – Magnification

- Texture smaller than surface – texture scaled up (magnification)
  - Nearest neighbor filtering – assigns fragment the color of the closest texture point
  - Linear filtering – Weighted average of four closest texture point



Nearest Neighbor



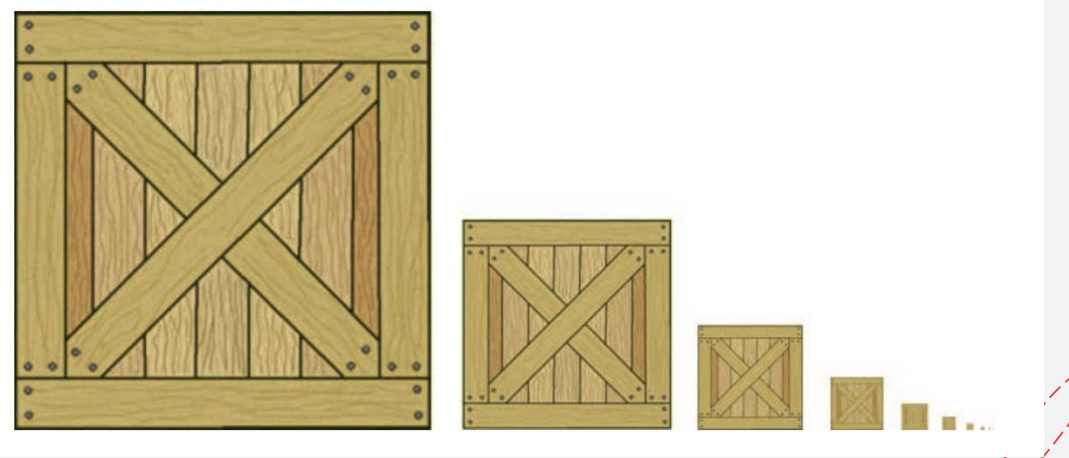


GL_NEAREST

GL_LINEAR

Scaling Image Data – Minification

- Texture larger than surface area – texture scale down (minification)
  - Many texture points overlap each fragment
  - Nearest neighbor filter creates visual artificats
  - Linear filtering highly inefficient
  - Sample from a precomputed *mipmai* – sequence of images, each half the size



Wrap Texture

- We could set up how to wrap a texture using ***glTexParameteri()***
  - Parameter names:
    - GL_TEXTURE_WRAP_S, GL_TEXTURE_WRAP_T
  - Parameter values:
    - GL_REPEAT, GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER

OpenGL Functions

- Similar to vertex buffers and attribute data

- ***glGenTextures****(textureCount)*
  - Return a set of available texture references

- ***glBindTexture****(bindTarget, textureRef)*
  - Binds referenced texture to target
  - Future operations affecting target will be applied to referenced texture

OpenGL Functions

- ***glTexImage2D****(bindTarget, level, internalFormat, width, height, border, format, type, pixelData)*
  - Create storage and upload *pixelData*
  - *width/height* – Image Dimensions
  - *border* – how thick the border is around the image, usually 0
  - *level* – level in a mipmap, usually 0
  - *format* and *internalFormat* – what format the pixel data has, usually GL_RGB or GL_RGBA
  - *type* – precision of color data, usually GL_UNSIGNED_BYTE

More OpenGL functions

- ***glGenerateMipmap****(bindTarget)*
  - Generates a sequence of mipmap images
  - This only works when images have been uploaded to GPU

- ***glTexParameteri****(bindTarget, parameterName, parameterValue)*
  - *parameterName:*
    - GL_TEXTURE_MAG_FILTER, GL_TEXTURE_MIN_FILTER, etc
  - *parameterValue:*
    - GL_NEAREST, GL_LINEAR, GL_NEAREST_MIPMAP_LINEAR, etc.

Texture (UV) Coordinates

Texture/UV Coordinates

- Specify which points of an image correspond to which vertices of a geometric object
- Coordinates ranges from 0 to 1
  - (0,0) – Lower-left (1,1) – Upper-right
- Values are passed from vertex shader to fragment shader (interpolated)

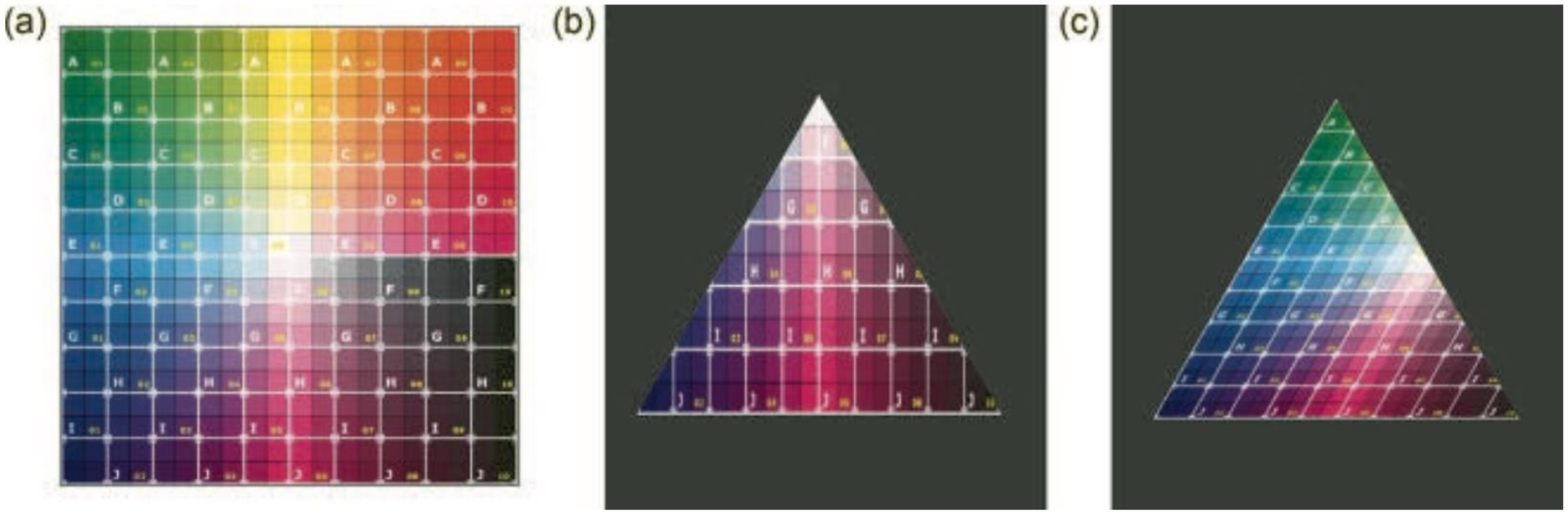
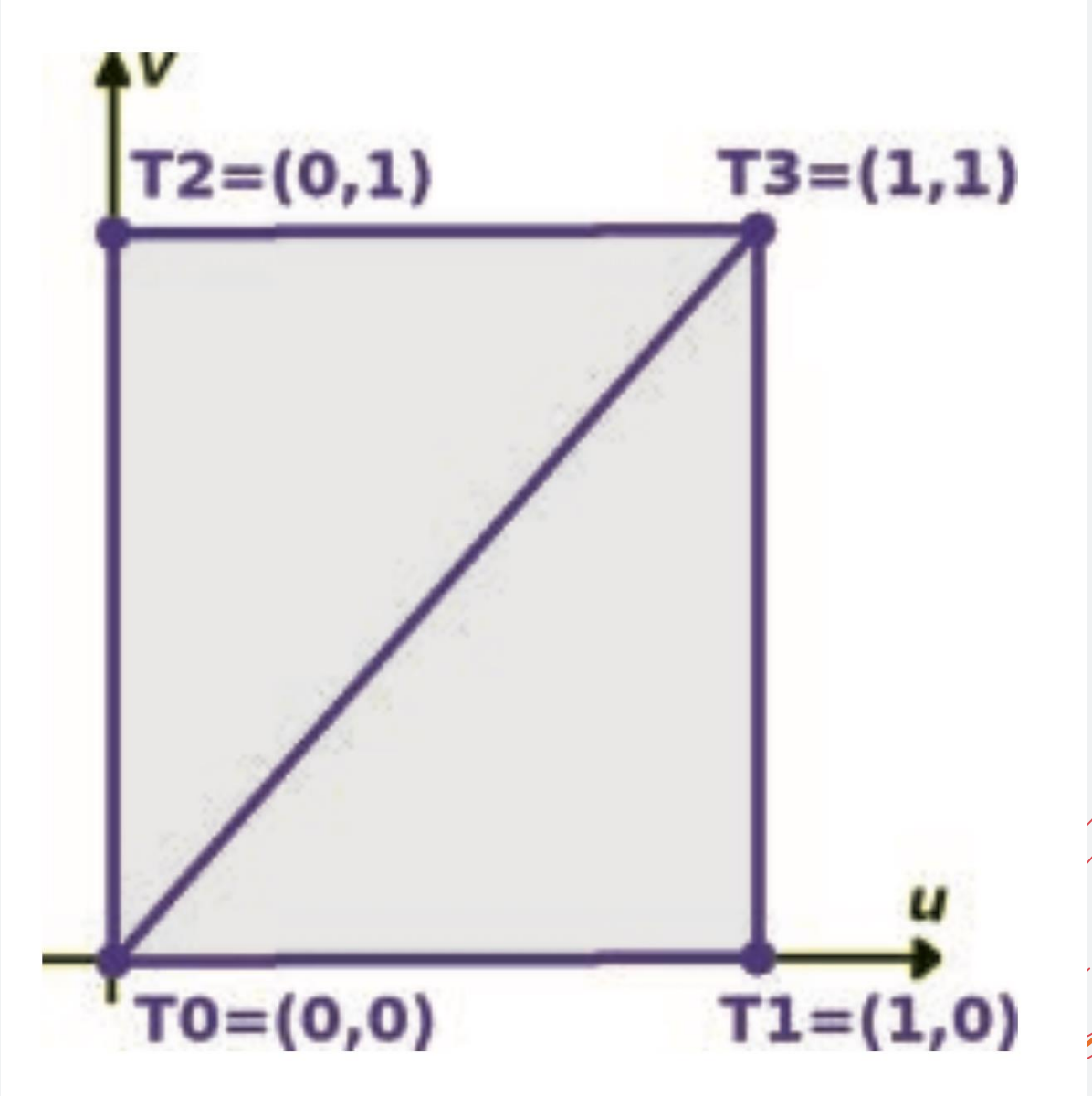


FIGURE 5.5 Texture coordinates used to apply an image (a) to a triangle (b, c).

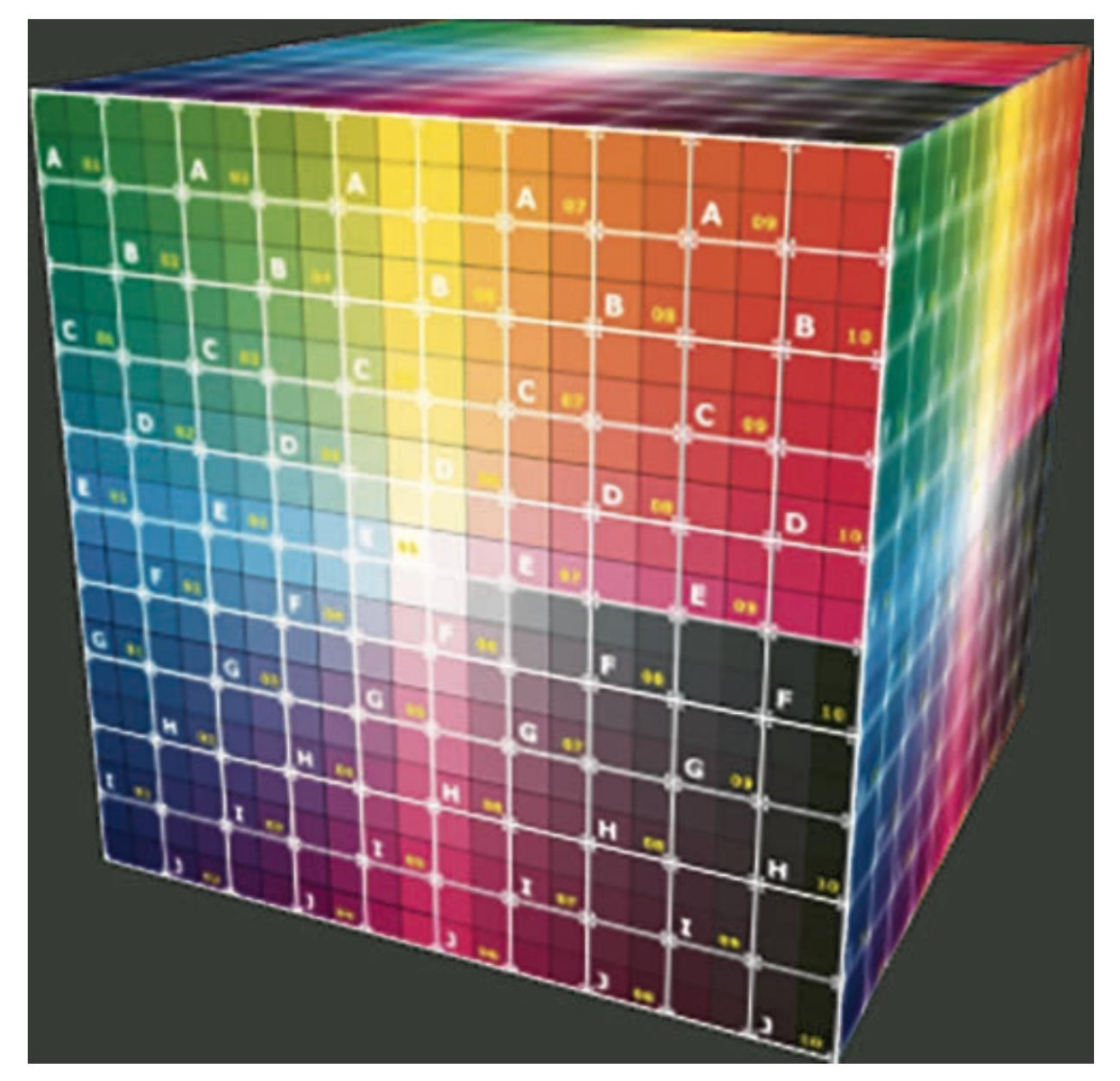
Rectangles

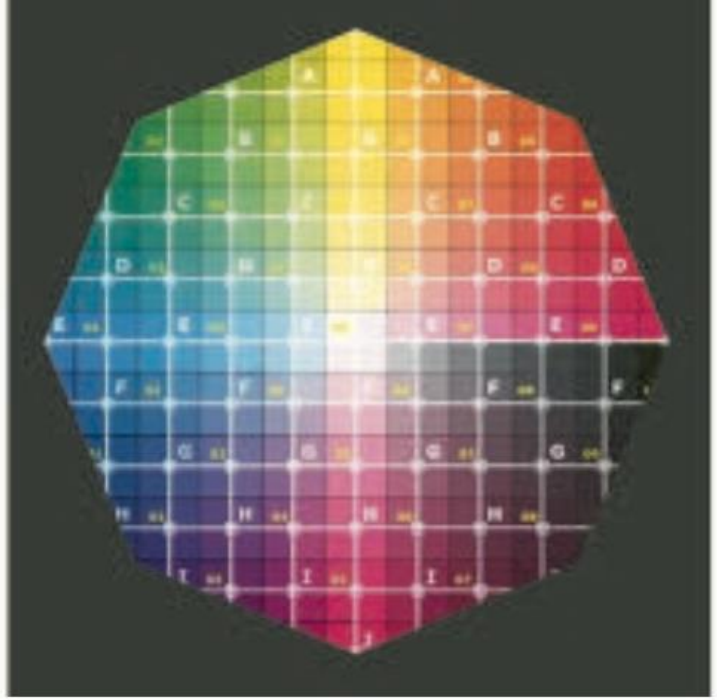
- A rectangle is made up of two triangles
- We want to associate the entire full texture image to a rectangle
- Make sure to assign texture coordinates in the same order that we assign position coordinates
  - T0, T1, T3
  - T0, T3, T2

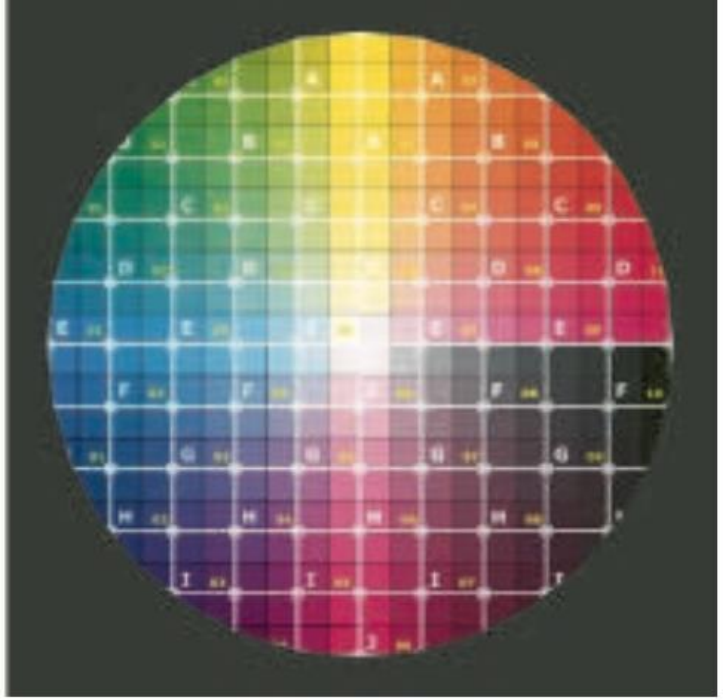


Boxes

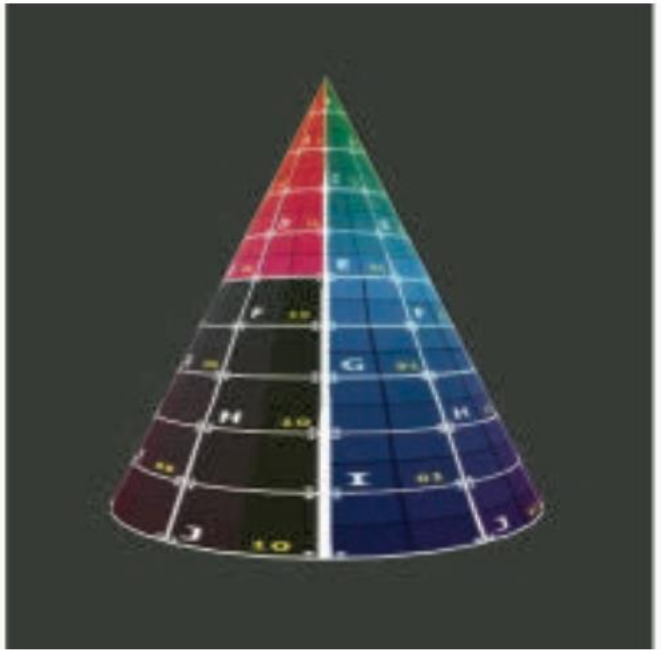
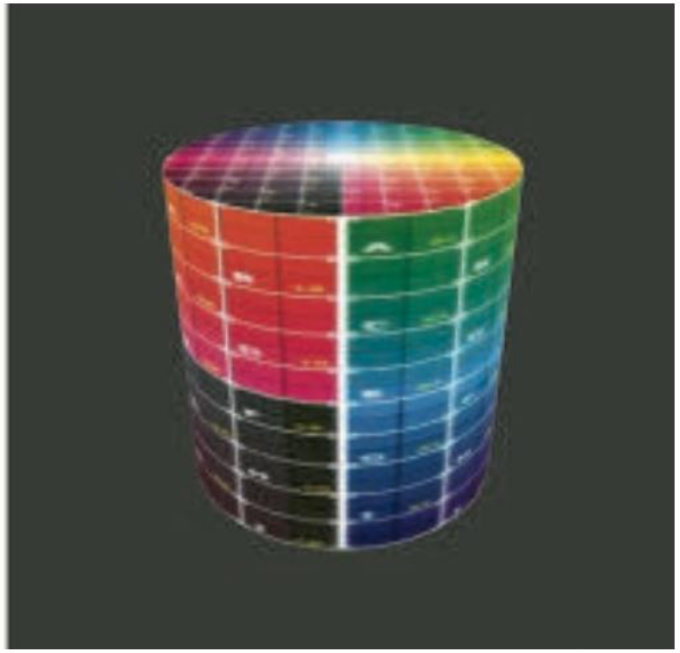
+ A box is made of six rectangles







Polygons

- Pretend a polygon is cutting out part of the texture
  - WON’T try to map the entire texture inside of one polygon

Parametric Surfaces